

Brechas de Implementación — Sistema de Tracking de Remediación

Nota de alcance

Este documento compara el estado real de **las tres capas del proyecto** (backend en `backend/src/main/java`, frontend en `frontend/src`, y el pipeline de ingestión en `workflows/` sobre n8n) contra el marco conceptual de `Tracking_Remediacion_Documento_Unificado.md` (mismo directorio), incorpora los hallazgos de `caja_negra_caja_blanca_ciberseguridad.md` y consolida el análisis previo de `Requisitos_Tracking_Remediacion_Solido.md`. Una primera versión de este documento solo había relevado el backend a fondo; esta versión corrige varios hallazgos a la luz de lo que realmente hace n8n y de lo que ya existe en la UI (ver notas marcadas **[corrección]** más abajo).

No es una auditoría de seguridad — es un análisis de **madurez del modelo de tracking**: qué tan defendible sería afirmar, ante un lector técnico exigente, que este sistema "hace seguimiento" de vulnerabilidades y no solo "las lista".

Las secciones están ordenadas de mayor a menor importancia real para el proyecto — no siguen el orden en que el marco de referencia presenta sus propios capítulos. Lo más crítico (qué implementar y por qué) va primero; el detalle de evidencia que respalda cada afirmación, las aclaraciones de alcance y lo explícitamente descartado quedan al final, como material de consulta.

1. Resumen ejecutivo

Nivel de madurez de verificación actual (Sección 10.1 del marco): entre 1 y 2, con un mecanismo automático que aspira a nivel 3 pero está roto por un antipatrón conocido.

- El cierre manual (analista mueve el Kanban a RESUELTA) es **nivel 1 — declarativo**: se estampa una fecha porque una persona lo dijo, sin comprobación.
- El cierre automático por ausencia en el escaneo siguiente *se parece* a una verificación de nivel 3, pero **no chequea que el escaneo haya tenido cobertura completa antes de cerrar** — es exactamente el antipatrón "cierre por ausencia" que el marco señala como el más peligroso de todos (Sección 7.1.1), porque produce métricas que mejoran a medida que la cobertura empeora, de forma silenciosa.
- El componente que el propio marco identifica como el que "vuelve auditable a todo el resto" — el **registro de auditoría inmutable de transiciones de estado** (Componente F) — **no existe**. Hay un servicio llamado `VulnerabilityAuditService` y una entidad `VulnerabilityAudit`, pero ninguno de los dos registra *quién cambió qué estado, cuándo y por qué*; registran hallazgos de escaneo, no decisiones humanas.
- El campo `reopenCount`, que sería la base de la tasa de recurrencia (una métrica de primer orden según el marco), se incrementa pero **no se lee en ningún lugar**: ni en el dashboard, ni en el DTO expuesto al frontend.
- No existe ningún campo de plazo/SLA. La priorización llega ya calculada desde el pipeline externo (n8n) — **[corrección]** y ese cálculo es más sólido de lo que parecía: n8n combina CVSS, criticidad del entorno (`Environment.businessCriticality`), el catálogo CISA KEV (con match por CVE) y condición de zero-day. El backend Java solo almacena el resultado final; no recalcula nada, pero tampoco hace falta que lo haga desde cero — el gap real es que **no persiste las variables que produjeron ese resultado**, solo la etiqueta final (ver Componente C, Sección 3).
- El "MTTR" que hoy muestra el dashboard es, en la terminología exacta del marco, **MTTR nominal** (mide hasta la marca manual), no MTTR estricto — y el propio comentario del código lo llama "MTTR real", lo cual es el antipatrón "MTTR nominal" de la Sección 19 aplicado literalmente.
- De los **10 KPI mínimos de MVP** que el marco recomienda para un dashboard operativo (Sección 14.5), solo **1 está implementado sin reservas** (distribución por severidad); el resto está parcial, mal nombrado o directamente ausente (ver Sección 4).
- [hallazgo nuevo, arquitectura]** n8n no solo alimenta al backend por webhook: el nodo `Persist_Audit_Findings` escribe **directo a la tabla** `vulnerabilities_audit` de Postgres con un `INSERT` propio, sin pasar por ningún endpoint del backend. El webhook `/api/webhook/scan-report` solo manda estadísticas agregadas del scan, nunca los hallazgos individuales. Esto importa para cualquier mejora futura: enriquecer esa tabla (rangos GHSA, evidencia, lo que sea) requiere tocar la query de n8n, no (solo) el backend Java.
- [hallazgo nuevo]** Ya existe una señal de cobertura parcial que llega al backend hoy — `ScanReport.systemStatus` guarda el string `" ⚠️ PARCIAL (GitHub Advisory falló)"` cuando n8n no pudo consultar el advisory de algún ítem — pero `applyLifecycle()` no la consulta antes de cerrar por ausencia. El fix del antipatrón "cierre por ausencia" (Componente A) es más barato de lo que se pensaba: no requiere esperar a que n8n mande cobertura nueva, alcanza con leer un dato que ya está llegando.

Lo que sí está bien resuelto (para no partir de cero en la lectura): identidad de activo estable y no volátil, reconciliación update-or-create correcta, separación deliberada entre estado automático y estado de gestión humana, no-borrado del par componente-CVE al cerrar, el

cálculo de prioridad multi-variable en n8n, y el selector de responsable en el Kanban (ya restringido a usuarios reales, no texto libre) — ver Sección 9.

2. Qué implementar, en orden de prioridad

Orden pensado por dependencia técnica real (igual que la Sección 7 del marco: cada componente necesita al anterior) y por costo/beneficio dentro de cada nivel de dependencia — lo más barato de resolver con datos que ya existen va primero dentro de cada bloque.

- Componente F — Auditoría de transiciones.** Sin esto, nada de lo que sigue es verificable. Tabla nueva de transiciones (`from_state` , `to_state` , `occurred_at` , `actor_id` , `actor_type` , `trigger` , `evidence_ref`), poblada tanto en el cambio manual de Kanban como en las transiciones automáticas de `applyLifecycle()` . Conviene resolver junto con esto la separación entre "hallazgo persistente" y "ciclo de remediación" (ver Componente E en la Sección 3): son la misma decisión de modelado, y hacerlo por separado después obliga a rehacer las consultas de métricas ya construidas.
- Componente A — Cierre consciente de cobertura.** Antes de tocar la lógica de verificación, arreglar que `applyLifecycle()` no cierre por ausencia si el escaneo fue parcial/fallido. **Más barato de lo que parece:** n8n ya setea `ScanReport.systemStatus = "⚠️ PARCIAL (GitHub Advisory falló)"` cuando la consulta a GHSA falla para algún ítem, y ese dato ya llega al backend hoy — alcanza con que `applyLifecycle()` lo consulte antes de cerrar, sin esperar ningún cambio en n8n. Nota: esa señal cubre solo "falló la consulta a GitHub", no "el activo nunca se escaneó" — para cobertura completa (`assets_in_scope` / `assets_analyzed`) sigue haciendo falta que `ScanReport` empiece a registrarla y que n8n la envíe, y los estados `PARTIALLY_COMPLETED` / `FAILED` siguen sin usarse en código ejecutable (solo en comentarios) — pero eso puede quedar como una segunda vuelta, no bloquea el primer fix.
- Exponer `reopenCount`** en el DTO y calcular tasa de recurrencia en `DashboardService` . Esfuerzo mínimo, el dato ya existe y solo está excluido del mapper.
- Separar MTTR nominal de MTTR verificado** en el dashboard, y declarar explícitamente el ancla de cada métrica de tiempo mostrada. De paso, agregar los dos KPI de la Sección 14.5 que quedan mecánicamente disponibles con lo que ya existe: **aging de vulnerabilidades** (bucketizar `firstDetectedAt` en 0-30/31-60/61-90/+90 días, sobre lo que ya está abierto) y **tiempo promedio por criticidad** (el mismo query de `mttrDays` , agrupado por `priority` en vez de global).
- Estados VEX** (mitigada / no aplica / riesgo aceptado con caducidad) reemplazando el campo `decision` de texto libre.
- Persistir las variables que produjeron la prioridad, no solo el resultado final. [corregido]** No hace falta "incorporar" `Environment.businessCriticality` al cálculo — n8n (`Risk_Assessment_Engine`) ya lo hace, junto con CVSS, el catálogo CISA KEV y condición de zero-day, y ya genera un `action_plan` textual con la recomendación. Lo que falta es que el backend **guarde esas variables** (equivalente al campo `priority_variables` del modelo de referencia, Sección 16 del marco) en vez de solo la etiqueta `priority` final — sin esto, nadie puede auditar después por qué un caso quedó CRITICAL. Requiere que n8n mande ese desglose en el payload (hoy no lo manda) y que el backend lo persista.
- Campo de plazo/SLA**, derivado del desglose de variables del punto 6 (ya existe la base de cálculo — CVSS + entorno + KEV + zero-day —, falta convertirlo en una fecha límite y no solo en una etiqueta de prioridad).
- Escala de evidencia E0-E6** (o una versión simplificada) para clasificar la fuerza de cada cierre, **y un campo para capturar o referenciar el respaldo real** (URL de reporte de escáner, commit, PR) — no alcanza con solo etiquetar el nivel de evidencia si no hay dónde guardar la evidencia misma. Hoy no existe ningún campo de este tipo en ningún modelo, ni ningún componente de adjuntos en la UI (confirmado por grep dirigido en backend y por revisión directa del modal de gestión de vulnerabilidad en el Kanban).
- Enriquecer `GhsaAdvisoryCache`** con rango estructurado y campo `reviewed` / `unreviewed` — este último requiere tocar también el pipeline de n8n (confirmado: n8n nunca pide ni parsea esos campos hoy, ver detalle y nota de arquitectura en la Sección 6). **Este ítem no está completo si se queda solo en el modelo de datos:** hay que además construir la lógica de comparación versión-vs-rango en `VulnerabilityAuditService` (el pseudocódigo de la Sección 10.2 del marco) que reemplace o complemente el cierre por ausencia de `applyLifecycle()` . Enriquecer la cache sin escribir esta comparación no mueve la aguja de madurez — es el paso concreto que separa el nivel 2 del nivel 3 de verificación (Sección 10.1 del marco), y es el único ítem de toda esta lista que, si se omite, invalida la afirmación de "tracking sólido" aunque se implemente todo lo demás.
- Normalizar `assignedTo`** a una relación con `User` /equipo en el backend (hoy es columna `String` sin FK). **[corregido]** La urgencia es menor de lo que parecía: la UI del Kanban ya restringe la asignación a un selector de usuarios reales (`userApi.getAll()`), no a texto libre — el gap es solo de integridad referencial en el esquema, no de experiencia de usuario.
- Campo `exposed` (booleano, manual) en `Asset`** (ver Sección 8) — solo si además de la criticidad del entorno (punto 6) hace falta distinguir específicamente "alcanzable desde internet". No es necesario hacerlo junto con el punto 6; es una variable distinta y de menor prioridad.

¿Implementar los 11 puntos alcanza un tracking "sólido"? Sí, en los términos que el propio marco define: llega al **nivel 3 de la escala de madurez de verificación** (Sección 10.1) — el salto cualitativo que el marco describe como el que separa "un gestor de tareas con fechas" de "un tracking de remediación real", porque el sistema pasa a comprobar por sí mismo en vez de confiar en una declaración humana. Ese era el objetivo explícito con el que se armó esta lista (heredado de `Requisitos_Tracking_Remediacion_Solido.md`), no una casualidad. Lo que la Sección 11 deja fuera de alcance (nivel 4, deduplicación multi-fuente, EPSS/SSVC completos, autoevaluación SANS, anclaje normativo argentino, caja negra real) son mejoras de madurez *superior* a "sólido", no requisitos para llegar ahí — omitirlos no

invalida la afirmación de nivel 3. La única forma de implementar todo esto y **no** llegar a nivel 3 sería hacer el ítem 9 solo a medias (enriquecer la cache de GHSA sin construir la comparación versión-vs-rango que ahora ese mismo ítem deja explícita) o saltar el ítem 1 (sin Componente F, ninguna de las mejoras de los demás es auditable, aunque técnicamente "funcionen").

3. Detalle por componente, en el mismo orden de prioridad

Reordenado $F \rightarrow A \rightarrow E \rightarrow B \rightarrow C \rightarrow D$ (no alfabético como en el marco): así es como realmente conviene abordarlos, según la Sección 2.

Componente F — Auditoría

Es la brecha estructural más grande. Requiere una tabla nueva (`from_state` , `to_state` , `occurred_at` , `actor_id` , `actor_type` , `trigger` , `evidence_ref`) poblada en cada `updateStatus()` y en cada transición automática de `applyLifecycle()` . Sin esto, cualquier mejora en B, C, D o E queda sin forma de auditarse retroactivamente.

Componente A — Identidad y cobertura

Base sólida, con dos huecos concretos: 1. El cierre por ausencia (`VulnerabilityAuditService.applyLifecycle()` , líneas 337-348) debe condicionarse a que el escaneo haya sido exitoso/completo — hoy cierra incondicionalmente. **[corregido]** Ya existe una señal barata para esto: `n8n` setea `ScanReport.systemStatus = " ⚠️ PARCIAL (GitHub Advisory falló)"` cuando la consulta a GHSA falla para algún ítem, y ese valor ya llega al backend — solo falta que `applyLifecycle()` lo lea antes de cerrar. 2. Falta el registro de cobertura *completa* (`assets_in_scope` , `assets_analyzed` , `status` real distinto de siempre-COMPLETED) en `ScanReport` — esta sí requiere que `n8n` empiece a mandar ese dato. Es una segunda vuelta más cara, no bloquea el punto 1. 3. La query de `n8n` que arma el universo del scan (`Check_Internal_Assets`) ya filtra `WHERE ... deleted_at IS NULL` en ambas tablas — confirma en código real el antipatrón "activo dado de baja se cierra por ausencia" (Sección 7.1 del marco): un activo dado de baja queda fuera del scan siguiente y sus hallazgos abiertos se cerrarían solos, sin distinguirlo de una remediación real.

Componente E — Recurrencia

El dato (`reopenCount`) ya existe y solo falta exponerlo — es la brecha de menor esfuerzo de todo el documento. Separar el "ciclo de remediación" del "hallazgo persistente" como entidades distintas es más costoso pero es lo que evita perder la duración de ciclos anteriores en cada reapertura (hoy `AssetVulnerability` sobrescribe `resolvedAt` a `null` en cada reapertura, `updateStatus` , línea 128). Conviene resolverlo junto con el Componente F: son la misma decisión de modelado (eventos como entidad de primer orden).

Componente B — Estados y evidencia

Falta desdoblar el timestamp único de resolución en los momentos que el marco distingue: reconocimiento, inicio de remediación, declaración, verificación. Falta un estado de mitigación y uno de "no aplica" con justificación estructurada — hoy todo eso vive en un campo de texto libre (`decision`).

Componente C — Plazos

No existe ningún campo de plazo. Es la brecha más simple de cerrar técnicamente (un campo calculado), pero requiere primero decidir el criterio.

[corregido] El criterio de priorización ya es multi-variable — no hace falta diseñarlo, hace falta persistirlo. La primera versión de este documento decía que `Environment.businessCriticality` (`Environment.java:19-20` , sembrado por `seed: Testing → MEDIA` , `Producción → CRITICA`) "no lo lee ningún cálculo de prioridad" — eso es cierto solo para el código Java. El nodo `Risk_Assessment_Engine` de `n8n` ya lo lee (vía la query `Check_Internal_Assets` , que trae `e.business_criticality AS env_criticality`) y lo combina con CVSS, el catálogo CISA KEV (`Fetch_CISA_KEV` , `match` por CVE) y condición de zero-day para producir la `priority` final, más un `action_plan` textual. Es, en la práctica, un acercamiento razonable al modelo de variables múltiples que pide la Sección 9 del marco — solo que ocurre en `n8n`, no en el backend, y **el backend solo guarda la etiqueta resultante, nunca las variables que la produjeron** (el `priority_variables` del modelo de referencia, Sección 16). Ese es el gap real de este componente: no calcular prioridad desde cero, sino hacer auditable la que ya se calcula.

Componente D — Verificación

El mecanismo de cierre automático necesita dos cosas para acercarse de verdad al nivel 3: (1) el chequeo de cobertura de A, y (2) que `GhsaAdvisoryCache` almacene el rango afectado estructurado (no solo texto de presentación), para poder comparar versión contra rango en vez de inferir por ausencia. Se prioriza después de B y C porque enriquecer `GhsaAdvisoryCache` requiere tocar también el pipeline externo de `n8n` — es la dependencia más cara de las seis.

4. Los 10 KPI mínimos de MVP (Sección 14.5 del marco), verificados uno por uno

Releí `DashboardService.java` completo (única fuente de KPIs del backend — `ReportPdfService` solo reimprime el mismo `DashboardStatsDTO` en PDF, no agrega ninguno nuevo) y grepeé cada repositorio de vulnerabilidades por los nombres de query relevantes (`countGroupedBy*` , `aging` , `backlog` , `assignedTo`) para no dar por sentado lo que un relevamiento previo había resumido.

#	KPI (Sección 14.5)	Estado	Evidencia
1	MTTR (con ancla declarada)	⚠ Existe, pero es nominal y sin ancla declarada	<code>mttrDays</code> (<code>DashboardService.java:57/119</code>), fórmula en <code>AssetVulnerabilityRepository.findAverageMttrDays()</code> .
2	Cumplimiento de SLA	❌ No existe	No hay campo de plazo en ningún modelo (ver Componente C) — no hay nada contra qué medir cumplimiento.
3	Backlog de vulnerabilidades	⚠ Parcial	<code>openVulnerabilities</code> da el total abierto (<code>DashboardService.java:53-54</code>), pero no desglosado por severidad/proyecto ni con tendencia histórica de backlog — solo el número actual.
4	Vulnerabilidades abiertas vs. cerradas	⚠ Parcial	<code>trendsLast30Days</code> (detecciones y resoluciones diarias, <code>DashboardService.java:97-111</code>) cubre esto, pero acotado a una ventana fija de 30 días, no un balance mensual configurable.
5	Aging de vulnerabilidades (0-30/31-60/61-90/+90 días)	❌ No existe	Sin grep positivo para <code>aging</code> / <code>backlog</code> en ningún repository — no hay ningún query que agrupe por antigüedad.
6	Tiempo promedio por estado	❌ No existe	Requiere el Componente F (historial de transiciones), que no existe.
7	Vulnerabilidades por severidad	✅ Sí	<code>severityDistribution</code> / <code>countGroupedByPriority()</code> (<code>AssetVulnerabilityRepository.java:86-91</code>).
8	Vulnerabilidades por responsable	❌ No existe	Sin grep positivo para ningún <code>GROUP BY</code> / <code>countGroupedBy</code> sobre <code>assignedTo</code> en <code>AssetVulnerabilityRepository</code> ni <code>VulnerabilityAuditRepository</code> — el campo existe (texto libre, ver Sección 2 ítem 10) pero nunca se agrupa por él.
9	Tiempo promedio por criticidad	❌ No existe	Solo hay un <code>mttrDays</code> agregado general (<code>DashboardService.java:57</code>); ningún query lo desglosa por <code>priority</code> .
10	Reincidencia de vulnerabilidades	❌ No existe	<code>reopenCount</code> se incrementa pero no se lee en ningún query de <code>DashboardService</code> (ver Componente E, Sección 3).

Resultado: 1 de 10 implementado sin reservas (severidad), 2 parciales (backlog total, tendencia 30 días), 1 con nombre incorrecto (MTTR), y 6 directamente ausentes. El frontend (`Dashboard.tsx:48-50`) confirma que no hay cómputo adicional del lado del cliente — solo reproduce los cuatro campos que ya vienen del backend (`resolvedThisMonth` , `mttrDays` , `systemStatus` , más las distribuciones).

Sobre el catálogo complementario de la Sección 13.3 (tiempo hasta asignación, tasa de remediación, riesgo residual, vulnerabilidades por activo, distribución por CWE/OWASP, % falsos positivos, etc.): no se verificó ítem por ítem con la misma exhaustividad que la tabla de arriba. Confirmé por grep dirigido que no existe ningún campo `cwe` / `owasp` / `falsePositive` en el modelo, lo cual respalda que ese catálogo secundario también está ausente, pero no se revisó cada ítem individualmente — queda pendiente si se necesita precisión total sobre ese catálogo.

Métricas — chequeo general (además del KPI por KPI de arriba):

Pregunta	Estado
¿Existe el timestamp que cada fórmula requiere?	⚠ Parcial — existen 4 de los 8 timestamps del ciclo completo (Sección 5.1 del marco).
¿Está declarado el ancla del intervalo?	❌ No — el dashboard expone <code>mttrDays</code> sin indicar desde/hasta qué evento se mide.
¿Está declarado el denominador de cada proporción?	❌ No — no hay proporciones con denominador declarado (no hay SLA compliance, no hay aging).
¿Está declarada la familia de cada métrica (actividad/resultado/riesgo)?	❌ No
¿El nombre de la métrica corresponde a lo que mide?	❌ No — <code>mttrDays</code> mide tiempo de gestión (primera detección → marca manual RESUELTA), pero el propio comentario del código lo llama "MTTR real" (<code>AssetVulnerabilityRepository.java:144-145</code>). Es el antipatrón "MTTR nominal" de la Sección 19, literal.

5. Checklist completo de la Sección 20 del marco (evidencia detallada de respaldo)

Esta sección es material de referencia — el detalle línea por línea que respalda las afirmaciones de las Secciones 1-4. Reordenada F → A → E → B → C → D para mantener consistencia con el orden de prioridad del resto del documento.

F — Auditoría

#	Pregunta	Estado	Evidencia
1	¿Cada transición genera un registro propio que no sobrescribe al anterior?	✗ No	<code>updateStatus()</code> solo hace <code>av.setUpdatedAt(...)</code> y sobrescribe la fila (<code>VulnerabilityAuditService.java:113-133</code>). No existe tabla de transiciones.
2	¿Se registra el actor y se distingue humano/escáner/política?	✗ No	No existe campo <code>actor_type</code> en ningún lado del modelo.
3	¿El historial sobrevive cuando el caso deja de estar activo?	⚠ N/A — no hay historial que sobreviva porque no existe	—
4	¿Se puede reconstruir el tiempo de permanencia en cada estado?	✗ No	Requiere el historial de transiciones, que no existe.

Este es el componente con más brecha total, y el marco es explícito en que es el que "vuelve auditable a todo el resto" — sin él, ninguna mejora en los otros cinco componentes es verificable ante un lector externo.

A — Identidad y cobertura

#	Pregunta	Estado	Evidencia
1	¿Clave de identidad del activo estable, distinta del nombre de presentación?	✅ Sí	<code>Asset.id</code> / <code>SoftwareComponent.id</code> autogeneradas, no dependen de IP/hostname.
2	¿Evita atributos volátiles (IP, hostname, id de instancia)?	✅ Sí	Ídem.
3	¿La identidad del hallazgo incorpora ubicación dentro del activo?	✗ No	<code>UNIQUE(software_component_id, cve_id)</code> (<code>VulnerabilityAuditService.java:275</code>) — sin tercer término. Dos ocurrencias del mismo CVE en rutas distintas del mismo componente colapsan en una fila. Salvedad: mientras la granularidad de detección siga siendo "un CVE por componente" (que es como opera hoy el pipeline de n8n), esta identidad compuesta ya es suficiente — no es prioritario hoy.
4	¿Regla de normalización externo→interno documentada y determinística?	⚠ Parcial	Existe la clave compuesta, pero no hay documentación explícita de la regla ni manejo de casos límite (ej. CVE=NULL con solo GHSA, ver nota de sesión anterior).
5	¿Actualiza el activo existente en lugar de insertar uno nuevo?	✅ Sí	<code>applyLifecycle()</code> — <code>existingByComponent</code> (líneas 276-282).
6	¿Se registra qué activos fueron efectivamente analizados en cada ejecución?	✗ No	<code>ScanReport</code> no tiene <code>assets_in_scope</code> / <code>assets_analyzed</code> / <code>sources_failed</code> .
7	¿Se distingue ejecución completa de parcial e impide cerrar con una parcial?	⚠ Parcial — señal disponible, no consultada	El campo técnico <code>status</code> contempla <code>PARTIALLY_COMPLETED</code> / <code>FAILED</code> solo en comentarios (<code>ScanReportDTO.java:22</code>); <code>ScanService.complete()</code> (línea 91) siempre escribe <code>"COMPLETED"</code> . Pero <code>ScanReport.systemStatus</code> (texto) sí recibe <code>"⚠ PARCIAL (GitHub Advisory falló)"</code> desde n8n cuando corresponde — el dato ya llega, solo que <code>applyLifecycle()</code> (líneas 337-348) no lo consulta antes de marcar <code>RESOLVED</code> .
8	¿Se conserva historial de versiones del activo?	✗ No	<code>SoftwareComponent.version</code> es un campo mutable único, sin tabla de historial.

E — Recurrencia

#	Pregunta	Estado	Evidencia
1	¿El par activo-advisory se conserva tras el cierre?	✅ Sí	<code>applyLifecycle()</code> reabre el mismo registro (líneas 310-325), no crea uno nuevo ni borra.

#	Pregunta	Estado	Evidencia
2	¿Estado de reapertura distinto del de detección inicial?	⚠️ Parcial	Vuelve a DETECTADA , el mismo estado que una detección nueva — no hay forma de diferenciar "es la primera vez" de "esto ya se había cerrado antes" sin mirar <code>reopenCount</code> .
3	¿Un mismo par acumula varios ciclos con timestamps independientes?	❌ No	Timestamps viven en el hallazgo persistente mismo (no hay entidad "ciclo" separada); cada reapertura sobrescribe <code>resolvedAt</code> .
4	¿Las decisiones de triage previas se re aplican al reaparecer?	❌ No	<code>triageStatus</code> se fuerza a DETECTADA , perdiendo cualquier decisión anterior registrada en <code>decision</code> .
5	¿La tasa de recurrencia se calcula y reporta?	❌ No	<code>reopenCount</code> existe (<code>VulnerabilityAuditService.java:324</code>) pero está excluido del DTO (<code>VulnerabilityAuditMapper.java:90</code> , <code>@Mapping(target = "reopenCount", ignore = true)</code>) y no aparece en ninguna query de <code>DashboardService</code> .

B — Estados y evidencia

#	Pregunta	Estado	Evidencia
1	¿Estado intermedio entre detección y resolución, con timestamp propio?	⚠️ Parcial	Existe <code>EN_ANALISIS</code> en <code>triageStatus</code> , pero sin timestamp propio (<code>acknowledgedAt</code> no existe).
2	¿Se distingue remediación declarada de verificada?	❌ No	Un único <code>resolvedAt</code> , estampado por marca manual (<code>updateStatus</code> , líneas 125-129) o por ausencia automática — ninguno de los dos casos es una verificación de rango de versión.
3	¿Se distingue remediación de mitigación?	❌ No	No existe estado de mitigación.
4	¿Descarte por "no aplica" con justificación obligatoria?	❌ No	Sin grep positivo para <code>no_aplica</code> / <code>falsePositive</code> en el modelo. Solo existe <code>decision</code> (texto libre, sin estructura).
5	¿Aceptación de riesgo con responsable y caducidad obligatoria?	❌ No	No existe en ningún modelo. Confirmado también por grep dirigido (<code>riskaccept</code> , <code>expiresat</code> , <code>caducidad</code>) sin resultados.
6	¿Cada transición estampa su propio timestamp, o se sobrescribe un campo único?	❌ Se sobrescribe	<code>resolvedAt</code> vuelve a <code>null</code> en cada reapertura (<code>updateStatus</code> , línea 128) — se pierde la duración del ciclo anterior.
7	¿Se registra el nivel de evidencia que respalda cada cierre?	❌ No	No existe escala E0-E6 ni campo equivalente, ni ningún campo para adjuntar/referenciar el respaldo real (URL de reporte, commit, PR) — confirmado por grep dirigido sin resultados.

C — Plazos

#	Pregunta	Estado	Evidencia
1-6 (todas)	¿Fecha límite, criterio documentado, variables, recálculo, detección de vencidos?	❌ No, en todos los puntos	Grep sin resultados para <code>due_date dueDate sla deadline</code> en <code>model/</code> , <code>service/</code> , <code>repository/</code> . Ninguno bloquea por falta de criterio: n8n ya calcula <code>priority</code> combinando CVSS + <code>businessCriticality</code> + CISA KEV + zero-day (ver Componente C en la Sección 3) — lo que falta es persistir esas variables y convertirlas en una fecha, no diseñar el criterio desde cero.

D — Verificación

#	Pregunta	Estado	Evidencia
1	¿El cierre requiere comprobación del sistema o basta la declaración humana?	⚠️ Mixto, ambos caminos con problemas	Manual = declarativo puro. Automático (por ausencia) = sin chequeo de cobertura.
2	¿El sistema compara versión entre escaneos consecutivos contra el rango del advisory?	❌ No, como comparación explícita en backend	El cierre es por <i>ausencia del CVE</i> , no por comparación estructurada <code>version_actual</code> vs. rango. <code>GhsaAdvisoryCache</code> ni siquiera almacena el rango afectado (solo <code>summary</code> / <code>description</code> / <code>referenceUrls</code>).
3	¿Se registra versión previa, versión posterior, método?	⚠️ Parcial	Existe <code>patchedVersion</code> como campo, pero sin trazabilidad de qué comparación lo produjo.
4	¿Se registra cuando la verificación falla (persiste pese a declararse resuelta)?	❌ No	Solo existe <code>reopenCount++</code> , sin distinguir "reapareció tiempo después" de "nunca se fue".
5	¿Reverificación periódica de casos cerrados?	❌ No	No se encontró <code>scheduler/cron</code> (<code>@Scheduled</code>) para reevaluar hallazgos cerrados — confirmado por grep en todo el backend.

6. GHSA como fuente

`GhsaAdvisoryCache` guarda únicamente `summary`, `description`, `referenceUrls`, `fetchedAt` — es un servicio de presentación (mostrar texto en el tablero), no de verificación. No distingue `reviewed` / `unreviewed`, no registra `withdrawn`, no modela rangos múltiples por paquete. `GhsaAdvisoryService` (líneas 44-96) descarta explícitamente esos campos al consumir la API de GitHub. Esto es una dependencia real del Componente D (Sección 3) — mejorar la verificación de nivel 3 requiere primero enriquecer esta cache.

Confirmado en n8n (`Fetch_GitHub_Advisories`): pega a `GET /advisories?ecosystem=X&affects=Y` y `Risk_Assessment_Engine` solo lee `first_patched_version` y el score CVSS de la respuesta — nunca pide ni parsea rangos completos, `withdrawn_at` ni estado de revisión, aunque la API de GitHub ya los devuelve en esa misma respuesta. Confirma que "enriquecer GHSA" sí requiere tocar n8n, no solo el backend.

Nota de arquitectura que afecta a este punto y a varios más de este documento: los hallazgos individuales no llegan al backend por API — el nodo `Persist_Audit_Findings` de n8n escribe **directo a la tabla `vulnerabilities_audit` de Postgres** con un `INSERT` propio armado con SQL. El webhook `/api/webhook/scan-report` que sí recibe el backend Java solo manda estadísticas agregadas del scan (totales, criticidad, `systemStatus`), nunca los hallazgos uno por uno. Cualquier campo nuevo en `vulnerabilities_audit` (rango GHSA, evidencia, lo que sea) requiere editar esa query de n8n, no solo la entidad Java correspondiente.

7. VEX / estados de descarte

No implementado. El único campo disponible (`decision`, texto libre) no distingue remediado / mitigado / no aplica / riesgo aceptado, y ninguno de esos estados exige justificación obligatoria ni fecha de caducidad. Confirmado también en el frontend: el Kanban (`kanban.tsx`) solo tiene tres columnas (`DETECTADA` / `EN_ANALISIS` / `RESUELTA`) — no hay ningún indicio, ni siquiera un placeholder visual, de mitigada/no-aplica/riesgo-aceptado. El único campo libre en el modal de gestión es un textarea de "Notas del analista" que mapea directo a `decision`.

8. Brecha estructural: el sistema es 100% Caja Blanca, sin ningún componente de Caja Negra

Esta sección aplica el segundo documento de referencia, `caja_negra_caja_blanca_ciberseguridad.md`, al proyecto — no es parte del marco de tracking, pero expone una limitación de alcance real que conviene documentar aparte porque afecta a uno de los componentes del tracking (ver más abajo). Se ubica hacia el final del documento porque, como se explica en el "matiz" más abajo, no requiere acción propia más allá de reusar un dato que ya existe.

Ese documento distingue dos aproximaciones para una plataforma de gestión de vulnerabilidades (su Sección 4-5):

- **Caja negra:** escaneo externo del activo — descubrimiento de host, puertos, servicios y versiones expuestas. Responde "*¿qué está expuesto desde afuera?*".
- **Caja blanca:** un agente instalado en el host que reporta sistema operativo, paquetes, dependencias y librerías instaladas. Responde "*¿qué software existe realmente dentro del activo?*".
- El documento recomienda **combinar ambas** para tener visión completa de la superficie de ataque, y lo ilustra con un diagrama explícito (Sección 5) donde escaneo externo y agente interno alimentan la misma correlación de vulnerabilidades.

GEF_Secure_App implementa únicamente la aproximación de caja blanca. El pipeline de n8n correlaciona el inventario declarado de dependencias (`SoftwareComponent` : paquete, ecosistema, versión, vía `url` /SBOM) contra advisories de GHSA — es exactamente el "agente interno" que describe la Sección 4.2 de ese documento. Confirmado revisando puntualmente `InventoryService`, `UrlParser`, `GhsaAdvisoryService` y `ScanService`, y con búsqueda dirigida sin resultados en todo `backend/src/main/java` para `nmap`, `puerto`, `portScan`, `banner`, `shodan`: no existe ningún mecanismo de descubrimiento externo, ni campos de puerto/servicio/banner, ni un `targetType` que represente "host escaneado desde afuera".

Por qué esto toca al tracking: el modelo de datos de `Asset` no tiene ningún campo de exposición — no existe `exposed` / `alcanzable_desde_internet`. El modelo de datos de referencia (Sección 16 del documento unificado) sí lo contempla, y la primera variable del modelo de cuatro variables de la BOD 26-04 (Sección 21.1) es exactamente esa pregunta — sin este campo, ni siquiera un criterio de priorización simple puede incorporarla.

Matiz — por qué esto es menos grave de lo que parece: el marco de tracking (Sección 6.1) trata al escaneo externo como *una fuente de detección entre varias* — el tracking no necesita generar sus propias detecciones, necesita gestionar bien las que recibe. Agregar caja negra real activaría de inmediato el problema de deduplicación multi-fuente (DefectDojo-style) que este mismo documento ya marca como no necesario hoy (Sección 11), justamente porque hoy solo hay una fuente (GHSA vía n8n).

Qué hacer con esto — de menor a mayor costo:

- **Más barato — ya está resuelto en la práctica:** `Environment.businessCriticality`, que n8n ya usa para ajustar la prioridad (ver Componente C, Sección 3), cubre buena parte de lo que "exposición del activo" busca resolver, sin agregar ningún campo nuevo — solo falta que el backend audite esa señal (ítem 6 de la Sección 2), no que la incorpore desde cero.
- **Intermedio:** si además hiciera falta distinguir específicamente "alcanzable desde internet", agregar un campo `exposed` (booleano) declarado manualmente en `Asset` (ítem 11 de la Sección 2).
- **Completo (grande, fuera de alcance recomendado):** implementar descubrimiento real de puertos/servicios (Nessus/OpenVAS/ `nmap`) e ingestarlo como una fuente más. Proyecto de infraestructura en sí mismo — ver Sección 11.

9. Fortalezas existentes (no perder de vista al implementar)

- Identidad de activo/componente estable, no basada en atributos volátiles.
- Reconciliación update-or-create correcta en la ingesta.
- Separación deliberada `detectionStatus` (automático) vs. `triageStatus` (gestión humana) — evita que el cierre automático oculte trabajo pendiente de revisión humana. Es una decisión de diseño más cuidadosa que lo que sugiere el resto del relevamiento.
- No-borrado del par componente-CVE al cerrar; se reabre en vez de duplicarse.
- Máquina de transiciones de `triageStatus` con reglas explícitas y validadas (aunque sin historial detrás).
- **Priorización multi-variable ya implementada en n8n** (`Risk_Assessment_Engine`): combina CVSS, criticidad del entorno, el catálogo CISA KEV (match por CVE) y condición de zero-day, y genera un `action_plan` textual con la recomendación concreta. Es, en espíritu, un acercamiento razonable al modelo de variables múltiples que pide la Sección 9 del marco — el backend solo necesita empezar a persistir ese desglose, no diseñarlo.
- **Asignación de responsable ya es un selector de usuarios reales en la UI** (`userApi.getAll()` en el Kanban), no texto libre — el gap de `assignedTo` es puramente de esquema (columna `String` sin FK), no de experiencia de usuario.
- `Environment.businessCriticality` **tiene CRUD completo en `Settings.tsx`** (selector BAJA/MEDIA/ALTA/CRITICA) y alimenta activamente el cálculo de prioridad en n8n — no es un campo sembrado y olvidado, es un dato mantenido y en uso real, solo que el backend Java no lo lee ni lo audita.

10. Seguridad de la plataforma misma (dominio adyacente, no parte del tracking)

Todo lo anterior asume que la plataforma que **gestiona** vulnerabilidades de terceros está, ella misma, bien asegurada. Es un dominio distinto — seguridad de la aplicación, no gestión de vulnerabilidades de activos externos — pero se documenta acá porque toca dos controles base de ciberseguridad (CIS Control 8, Audit Log Management; OWASP ASVS V7, Error Handling and Logging) que ninguna de las secciones anteriores cubre. Se ubica hacia el final porque es un dominio adyacente, no porque sea poco importante — el punto 10.2 (fuerza bruta) en particular es una exposición real y barata de cerrar.

10.1 No hay registro de eventos de autenticación

`AuthService.login()` (`AuthService.java:23-41`) no deja ningún rastro de los intentos de acceso, ni exitosos ni fallidos: no hay quién, cuándo, desde dónde, ni resultado. Es un registro distinto del Componente F (Sección 3): aquel es sobre transiciones de casos de vulnerabilidad; este es sobre accesos a la plataforma misma — ambos son necesarios, uno no reemplaza al otro.

10.2 Sin protección contra fuerza bruta en el login

No hay rate limiting ni bloqueo temporal tras varios intentos fallidos sobre `/api/auth/login` (la única ruta pública además de webhooks y docs, según `SecurityConfig.java:67`). Un atacante puede probar credenciales sin fricción ni límite.

10.3 Lo que ya está bien resuelto acá (no tocar)

`AuthService` usa BCrypt, devuelve el mismo mensaje de error para usuario inexistente y contraseña incorrecta (evita enumeración de usuarios), y el control de acceso por rol es real: `@PreAuthorize("hasRole(...)") / hasAnyRole(...)` está aplicado de forma consistente en prácticamente todos los controllers (`AssetController`, `UserController`, `VulnerabilityAuditController`, `EnvironmentController`, etc.). Esta sección señala dos puntos puntuales (10.1 y 10.2), no una debilidad general de la autenticación.

Nota para no confundir dos registros distintos: `Settings.tsx` tiene una pestaña de "log de errores" (`workflowId`, `nodeName`, `errorMessage`) que audita **fallas del pipeline de n8n**, no del Componente F (Sección 3) — aquel es sobre transiciones de estado de vulnerabilidades, este es sobre errores técnicos de ejecución de nodos. Son complementarios, no intercambiables, igual que 10.1 y el Componente F tampoco lo son.

11. Qué queda deliberadamente fuera de alcance

Para no perseguir un nivel de madurez que no es realista para este proyecto (el objetivo es un tracking *sólido*, no máxima madurez):

- **Nivel 4 de verificación** (escáner activo contra el sistema en ejecución) — requiere infraestructura de re-escaneo que excede el alcance actual.
 - **Deduplicación multi-fuente** (DefectDojo-style) — solo es relevante si se incorpora una segunda fuente de detección además de GHSA/n8n.
 - **Integración completa con EPSS/SSVC y el modelo formal de cuatro variables de la BOD 26-04** — mejora la calidad del criterio de priorización pero es un proyecto en sí mismo, y la directiva citada no es obligatoria fuera de agencias federales de EE. UU. **Corrección:** KEV **ya está parcialmente integrado** — n8n (`Fetch_CISA_KEV`) trae el catálogo completo y hace match por CVE dentro de `Risk_Assessment_Engine` para escalar prioridad. Lo que falta no es KEV en sí, sino persistir esa señal en el backend (ver Componente C, Sección 3) — no es un ítem 100% descartado como se decía antes.
 - **Autoevaluación SANS de madurez de programa** — es una capa organizacional, no de datos; no bloquea nada de lo anterior.
 - **Anclaje regulatorio argentino (CNC, Decisión 641/2021)** — relevante para un capítulo de marco normativo en la tesis, no para el modelo de datos del sistema.
 - **Componente de Caja Negra propiamente dicho** (escaneo de puertos/servicios, huella digital de red) — ver Sección 8. Confirmar la exposición real de un activo requiere infraestructura de escaneo externo que es un proyecto aparte, y activaría el problema de deduplicación multi-fuente ya descartado arriba; lo único que conviene resolver ahora es reusar `Environment.businessCriticality` (ítem 6 de la Sección 2) y, solo si hiciera falta más adelante, el campo manual `exposed` (ítem 11).
 - **location_key (ubicación dentro del activo)** — ver checklist A#3 en la Sección 5. La identidad (componente, CVE) alcanza mientras la granularidad de detección siga siendo un CVE por componente, que es como opera hoy el pipeline.
-

12. Una salvedad que no es código

La separación de funciones en la verificación — que quien verifica un cierre sea una persona distinta de quien lo remedió (Sección 6.8 del marco, rol "Verificador" en la Sección 17.1) — es una regla de **proceso**, no algo que el modelo de datos garantice por sí solo. Se puede agregar un campo `verified_by` a la tabla de transiciones del Componente F, pero que efectivamente lo llene una persona distinta del responsable de la remediación es una decisión operativa del equipo, no una brecha técnica que este documento pueda cerrar con una migración.